

[Clickup](#)

[Atomic] Explain Why Idle Vehicles Are Left Unused

1. Context

Today MVRO explains optimization gaps from the **job side** only: every task in `result.unassigned[]` carries a **reason** (`unmatched skills`, `insufficient capacity`, `time_window requirements`, `zone constraints`, `max_distance exceeded`, ...). What it cannot answer is the **vehicle-side** question operators actually ask when they look at a partial solution:

"I still have trucks sitting idle. Why couldn't any of them pick up the work that was left unassigned?"

When a solution contains both unassigned tasks **and** vehicles whose route is empty (no jobs, pickups, or deliveries assigned), the current output gives no link between the two. The operator has to manually cross-check every idle vehicle against every unserved task — skills, capacity, time windows, zones — to understand whether the idle vehicle was genuinely incapable, or whether leaving it idle was a tuning/cost decision.

This feature adds the **dual of the unassigned reason**: for each idle (empty-route) vehicle, an explanation of why it could not absorb the unassigned work. It is reported per vehicle as an aggregated set of hard-constraint reasons, plus a flag that distinguishes two fundamentally different situations:

- The idle vehicle is **incapable** of serving any unassigned unit — a hard constraint blocks every one (wrong skills, too little capacity, unreachable within time windows, ...).
- The idle vehicle **could** have served at least one unassigned unit, but the solver chose to leave both idle — a cost/objective decision, not a capability limit. This most commonly surfaces when `objective_penalties` lowers the unassigned penalty (see [Configurable Single-Objective with Layer Penalties](#)).

The output is **opt-in** and **additive**: existing responses are unchanged unless the new option is set.

2. API Changes

Scope

Route Optimization **Flexible API only** (nbro). The Fast engine (jsprit) **silently ignores** the option and never emits `unused_vehicles`, consistent with other flexible-only options such as `objective_penalties`.

Activation

A new optional boolean is added under `options`:

Options Object (additions)	Type	Required	Default	Remark
<code>report_unused_vehicle_reason</code>	bool	No	<code>false</code>	When <code>true</code> , populate <code>result.unused_vehicles[]</code> for idle vehicles whenever the solution has unassigned tasks.

When `false` or absent, the response shape is identical to today.

Definitions

- **Idle / empty-route vehicle** — an input vehicle whose final solution has **no** `job`, `pickup`, or `delivery` step. It either does not appear in `result.routes[]` at all, or appears with only `start/end` steps.
- **Unassigned unit** — a job, or a shipment treated as **one atomic unit** (pickup + all deliveries). Shipments are evaluated whole, never per-step.
- **Feasible (standalone)** — a vehicle `V` can serve unit `u` "feasibly" if `u` could be served on `V`'s otherwise-empty route without violating any hard constraint (skills, capacity/volume, time windows, zones, `max_distance`, `max_travel_time`, `max_tasks`, start/end reachability). The check is per-unit and independent; it does **not** consider serving multiple units together.

Output

A new top-level array `result.unused_vehicles[]`. It is present only when `report_unused_vehicle_reason` is `true` **and** `result.unassigned[]` is non-empty. If the option is on and unassigned tasks exist but every vehicle is used, the array is present and empty (`[]`).

<code>result.unused_vehicles[]</code>	Type	Description

<code>vehicle</code>	string	Idle vehicle ID.
<code>reasons</code>	array[string]	Distinct hard-constraint reasons, aggregated across all unassigned units this vehicle cannot serve. Empty if the vehicle can feasibly serve every unassigned unit.
<code>feasible_unassigned</code>	bool	<code>true</code> if the vehicle could serve at least one unassigned unit on its empty route. Marks "feasible but not selected": the idleness is a cost/objective outcome, not a capability limit.

`reasons` uses the **same vocabulary** as `result.unassigned[ ].reason`:

- `unmatched skills`
- `insufficient capacity`
- `time_window requirements`
- `zone constraints`
- `max_distance exceeded`
- `max_travel_time exceeded`
- `max_tasks exceeded`

Read the two fields together:

<code>reasons</code>	<code>feasible_unassigned</code>	Interpretation
non-empty	<code>false</code>	Vehicle is incapable of every unassigned unit; the listed constraints explain why.
non-empty	<code>true</code>	Vehicle is blocked from some units (listed) but could serve at least one other — that one was left unassigned by cost.

empty	true	Vehicle could serve every unassigned unit; all were left unassigned by cost/objective. Investigate tuning, not capability.
empty	false	Not emitted — a vehicle with no blocking reason and no feasible unit only occurs when there were no unassigned units.

Example — idle vehicle is incapable (default objective)

Two vehicles and three delivery jobs in Singapore. Combined single-run capacity is $60 + 40$, but demand and skill requirements make two jobs unservable. Vehicle $v-2$ ends up idle.

- $v-1$ carries skills $[1, 2]$, capacity 60 .
- $v-2$ carries skills $[1]$, capacity 40 .
- $job-1$ (high priority, 50 units, needs skill 2) fits only on $v-1$.
- $job-2$ (30 units, needs skill 2) needs skill $2 \rightarrow v-2$ cannot serve it; $v-1$ is already full ($50 + 30 > 60$).
- $job-3$ (45 units, needs skill 1) exceeds $v-2$'s capacity ($45 > 40$); $v-1$ is already full.

The solver assigns $job-1$ to $v-1$ (highest priority) and leaves $job-2$ and $job-3$ unassigned. $v-2$ is idle.

Request

```
{
  "description": "Idle vehicle reason demo",
  "locations": {
    "location": [
      "1.29068,103.85161",
      "1.28837,103.84470",
      "1.29360,103.80822",
      "1.30228,103.83640"
    ]
  },
  "vehicles": [
    {
      "id": "v-1",
      "start_index": 0,
```

```

    "end_index": 0,
    "skills": [1, 2],
    "capacity": [60]
  },
  {
    "id": "v-2",
    "start_index": 0,
    "end_index": 0,
    "skills": [1],
    "capacity": [40]
  }
],
"jobs": [
  {
    "id": "job-1",
    "location_index": 1,
    "delivery": [50],
    "skills": [2],
    "priority": 100,
    "service": 600
  },
  {
    "id": "job-2",
    "location_index": 2,
    "delivery": [30],
    "skills": [2],
    "service": 600
  },
  {
    "id": "job-3",
    "location_index": 3,
    "delivery": [45],
    "skills": [1],
    "service": 600
  }
],
"options": {
  "objective": { "travel_cost": "duration" },
  "report_unused_vehicle_reason": true
}
}

```

Response (abridged)

```

{
  "status": "Ok",
  "description": "Idle vehicle reason demo",
  "result": {
    "code": 0,

```

```

"summary": {
  "cost": 1800,
  "routes": 1,
  "unassigned": 2,
  "delivery": [50],
  "pickup": [0]
},
"routes": [
  {
    "vehicle": "v-1",
    "delivery": [50],
    "steps": [
      { "type": "start", "location_index": 0, "load": [50] },
      { "type": "job", "id": "job-1", "location_index": 1, "load": [0] },
      { "type": "end", "location_index": 0, "load": [0] }
    ]
  }
],
"unassigned": [
  { "id": "job-2", "type": "job", "reason": "insufficient capacity" },
  { "id": "job-3", "type": "job", "reason": "insufficient capacity" }
],
"unused_vehicles": [
  {
    "vehicle": "v-2",
    "reasons": ["unmatched skills", "insufficient capacity"],
    "feasible_unassigned": false
  }
]
}

```

The vehicle-side view adds information the job-side reasons cannot: both jobs are globally **insufficient capacity**, but for **v-2** specifically the blockers are **unmatched skills** (job-2 needs skill 2) **and insufficient capacity** (job-3 exceeds 40). **feasible_unassigned: false** confirms **v-2** is genuinely incapable — adding vehicles like **v-2** would not have helped.

Example — feasible but not selected (lowered unassigned penalty)

Starting from the same instance, add **job-4** (20 units, skill 1, at a distant, expensive location) and lower the unassigned penalty so the solver prefers to drop a costly job rather than serve it:

```

{
  "options": {
    "report_unused_vehicle_reason": true,
    "objective_penalties": { "unassigned_count_penalty": 100 }
  }
}

```

```

}
}
v-2 can feasibly serve job-4 (skill 1, 20 <= 40), but with unassigned_count_penalty
= 100 the transport cost of reaching job-4 exceeds the penalty, so the solver leaves both
job-4 and v-2 idle. The output now distinguishes this from a capability problem:

```

```

{
  "unused_vehicles": [
    {
      "vehicle": "v-2",
      "reasons": ["unmatched skills", "insufficient capacity"],
      "feasible_unassigned": true
    }
  ]
}

```

`feasible_unassigned: true` tells the operator that v-2 **could** have served unassigned work; the gap is a tuning decision, not a fleet-capability limit. See [Configurable Single-Objective with Layer Penalties](#) for why feasible jobs may be left unassigned under a unified objective.

Validation Rules

The following condition produces a **400 Bad Request**:

Condition	Error
<code>report_unused_vehicle_reason</code> is not a bool	<code>report_unused_vehicle_reason</code> must be a boolean

`report_unused_vehicle_reason: true` on the Fast engine is **not** an error; it is silently ignored (no `unused_vehicles` in the response).

Limitations

1. **Flexible only.** The Fast engine ignores the option.
2. **Empty-route vehicles only.** A vehicle that is used but still had spare capacity for an unassigned task is **not** reported — only fully idle vehicles appear in `unused_vehicles[]`.
3. **Standalone per-unit feasibility.** The feasibility check evaluates each unassigned unit independently on an empty route. It does not detect cases where a *combination* of units would have been jointly (in)feasible.
4. **Aggregated per vehicle.** `reasons` is a deduplicated set and is **not** linked to specific task ids. To find which task hit which constraint, cross-reference each unassigned task's own `reason`.
5. **Feasibility, not insertion order.** `feasible_unassigned` reflects hard-constraint feasibility on an empty route. Under heuristic search a feasible unit may, rarely, be missed even under the default objective; treat `feasible_unassigned: true` as "capability exists", not "solver bug".

6. **Reason quality inherits unassigned caveats.** Under a unified objective the exact `reasons` strings may degrade the same way `unassigned[ ].reason` does, but `feasible_unassigned` remains reliable.

Competitors

Google Route Optimization / OR-Tools expose per-shipment skip reasons but no first-class "why is this vehicle empty" diagnostic; operators reconstruct it client-side. OptaPlanner exposes score breakdowns but not vehicle-level idle explanations.

3. Tech Design

Producer

1. **Validate** `options.report_unused_vehicle_reason` as a boolean; default `false`. Echo the resolved value back under `options` for round-trip clarity.
2. **Gate** assembly on `flag == true` and `unassigned` non-empty. Otherwise omit `unused_vehicles` entirely (no empty array when the flag is off).
3. **Map** internal vehicle IDs back to the original `vehicles[ ].id` in the `unused_vehicles[ ]` output.
4. **Re-optimization.** `report_unused_vehicle_reason` is a request-level option; carry it through unchanged on each optimization call. No special handling.

Executor (NBRO)

After the search terminates, and only when the flag is set and at least one unit is unassigned:

1. **Collect idle vehicles** — input vehicles with zero `job/pickup/delivery` activities in the final solution.
2. **For each idle vehicle V , for each unassigned unit u** run a standalone hard-constraint feasibility check using the already-built travel matrix:
 - skills: `u.skills  $\subseteq$  V.skills`
 - capacity / volume: peak load of `u  $\leq$  V` capacity per dimension
 - time windows: a single-unit schedule (for shipments, pickup $\rightarrow$ delivery) fits within `V.time_window` and `u` time windows, including service/setup and `max_time_in_vehicle`
 - zones, `max_distance`, `max_travel_time`, `max_tasks`, start/end reachability
 - **Feasible** $\rightarrow$ set `V.feasible_unassigned = true`.
 - **Infeasible** $\rightarrow$ add the violated reason(s) to `V.reasons` (deduplicated).
3. **Emit** one `unused_vehicles[ ]` entry per idle vehicle with the aggregated `reasons` and `feasible_unassigned`.

Complexity is $O(|\text{idle vehicles}| \times |\text{unassigned units}|)$ cheap checks (set membership, scalar comparisons, a few matrix lookups). For pathological inputs the analysis

may be capped; if so, log the dropped count so support can tell a truncated diagnostic from a complete one — never silently truncate.

Fast

Not supported. The jsprit engine ignores `report_unused_vehicle_reason`; no executor changes.

Re-Optimization Interaction

Request-level option carried through each optimization call unchanged. The idle set and unassigned set are recomputed from the new solution every run.

4. Testing Suggestions

1. **Flag off (regression).** Existing flexible suite with the option unset — `unused_vehicles` must be absent, no solution diffs.
2. **Incapable vehicle.** The §2 main example. Expect one `unused_vehicles` entry for v-2 with `["unmatched skills", "insufficient capacity"]` and `feasible_unassigned: false`.
3. **Feasible but not selected.** The §2 second example with low `unassigned_count_penalty`. Expect `feasible_unassigned: true`.
4. **All-feasible idle vehicle.** Idle vehicle that could serve every unassigned unit (low penalty) → `reasons: [], feasible_unassigned: true`.
5. **No idle vehicles.** Unassigned tasks exist but all vehicles used → `unused_vehicles: []`.
6. **No unassigned tasks.** Flag on, full assignment → `unused_vehicles` absent.
7. **Shipment unit.** Unassigned one-to-one and one-to-many shipments evaluated as atomic units; a capacity block on the pickup amount yields one `insufficient capacity` reason for the whole shipment, not per-step.
8. **Time-window block.** Idle vehicle whose `time_window` cannot reach any unassigned task → `time_window requirements`.
9. **Zone / max_distance blocks.** Verify each maps to the matching reason.
10. **Mixed aggregate.** Idle vehicle blocked from some units by skills and from others by capacity → both reasons present, deduplicated.
11. **Validation.** Non-boolean `report_unused_vehicle_reason` → 400.
12. **Fast engine.** Flag `true` on jsprit → ignored, no `unused_vehicles`, no error.
13. **Re-opt.** Carry the flag across a re-optimization call; idle/unassigned sets recomputed from the new solution.

5. Notes

1. This feature is the **vehicle-side dual** of `unassigned[].reason`. Job-side reasons answer "why is this task unserved by anyone"; `unused_vehicles[]` answers "why is this specific idle vehicle not helping".

2. **Aggregate by design.** `reasons` is intentionally not linked to task ids (chosen for compactness). A future extension could add a per-reason `blocked_count` or task-id breakdown without breaking the current shape.
3. **Companion to objective tuning.** `feasible_unassigned: true` is the clearest signal that lowering `unassigned_count_penalty` (or other `objective_penalties`) caused the gap rather than the fleet — pair this output with when diagnosing support tickets.
4. **Only emitted with unassigned tasks.** A solution with idle vehicles but zero unassigned tasks does not populate `unused_vehicles[ ]` — idle-by-choice with full service is not a gap worth explaining.
5. **Documentation.** Update [api-reference.md](#) Result Response Schema with the `unused_vehicles[ ]` table and the `report_unused_vehicle_reason` option after implementation.

Clarifications Required from Engineering

1. Idle vehicles with no unassigned jobs?

The spec says `unused_vehicles` is omitted when there are no unassigned tasks. Should this remain true even if several vehicles are completely idle?

Engg: yes, `unused_vehicles` stays absent, by design.

The field explains one specific thing: why the idle vehicles could not take over the *unassigned* work. With zero unassigned tasks there is no gap to explain — vehicles left idle then simply mean the fleet is larger than the workload needs, which is a normal cost-optimal outcome, not a problem the diagnostic reports on. The full presence rules:

- flag off → **absent** (always)
- flag on, no unassigned tasks → **absent** (even if vehicles are idle)
- flag on, some unassigned, no idle vehicles → **[] (present but empty)**
- flag on, some unassigned, some idle vehicles → **populated**

The present-but-empty case is intentional, so "every vehicle was used" is distinguishable from "not applicable".

2. Re-optimization?

During re-optimization, are `unused_vehicles` recalculated from scratch every time, or can previous results influence them?

Engg: `report_unused_vehicle_reason` is a request-level option; each re-optimization computes the idle set, the unassigned set, and the reasons from that run's own solution.

Previous results never feed the diagnostic directly. They influence it only indirectly through what the re-opt request itself contains — for example, a vehicle kept busy by pinned routes from the parent result is not idle in the new solution, so it won't be reported.

3.Can an idle vehicle have different reasons from the unassigned jobs?

(For example, a job shows `insufficient capacity`, but the vehicle reports `unmatched skills`.)

Engg: yes, expected and consistent.

The two fields answer different questions:

- `unassigned[ ].reason` is job-side: why that task could not be served anywhere in the fleet.
- `unused_vehicles[ ].reasons` is vehicle-side: which hard constraints block *that particular idle vehicle* from the unassigned tasks — aggregated and de-duplicated across **all** unassigned tasks, not linked to specific task ids.

So "job says insufficient capacity, vehicle says unmatched skills" is a coherent picture: capacity was the fleet-wide blocker for that job, while this specific vehicle is (also or instead) blocked by skills — possibly by a different unassigned task. To map a specific task to a specific constraint, read the task's own `reason` together with the vehicle's list. The vehicle side also deliberately uses its own short vocabulary (`unmatched skills`, `insufficient capacity`, `time_window requirements`, `zone constraints`, `max_distance exceeded`, `max_travel_time exceeded`, `max_tasks exceeded`).

4.How does this feature interact with Split Orders? If a partially used vehicle has remaining capacity and an unassigned job is splittable, shouldn't the solver be able to split the job so that all vehicles are utilized and all work is assigned? In that scenario, would `unused_vehicles` still be expected?

Engg: Interaction with Split Orders — the solver does try exactly that, but per job it's all-or-nothing.

- Splitting runs as a dedicated final phase: any splittable job still unassigned after the main search is retried by dividing its load across the remaining capacity of already-used vehicles and idle vehicles alike — up to `max_splits` parts, at most one part per vehicle, each part at

least the **quant** share. So yes, a partially used vehicle's leftover capacity is precisely what splitting targets.

- However, a job is never partially fulfilled: either the entire load is placed across the parts, or the attempt is rolled back and the job stays unassigned in full. Every part must also satisfy all other hard constraints (time windows, skills, zones, ...), and serving all parts must still be cheaper than the unassigned penalty — so a splittable job can legitimately remain unassigned.
- Consequences for **unused_vehicles**: if splitting clears all unassigned work, the field is absent (rule from Q1). If a splittable job remains unassigned, the field is produced as usual — with one nuance: the idle-vehicle feasibility check evaluates each unassigned job at its full demand as one unit (splitting is not modeled inside the diagnostic), so an idle vehicle that could only carry a fragment still reports **insufficient capacity / feasible_unassigned: false**. That is consistent with the outcome, because the split phase has already tried — and failed — to place the job across the whole fleet, fragments included.
- Also remember: **unused_vehicles** reports completely idle (empty-route) vehicles only — a partially used vehicle never appears there, regardless of leftover capacity.